

Lab 2

Web Application Security

Ana-Maria Dobrescu, Aurora Navas Jimenez

April 28, 2026

1 Introduction

In this document, we will explain how we solved the lab. We will go through each of the vulnerabilities, answer the questions on the lab assignment, and explain how we patched them.

2 ReDoS + EXTRA Vulnerability

Description

To search for this vulnerability, we examined the code for user input processing that was not sanitized properly. The problem was found in the user registration logic, more specifically, in the *validatePassword* function. This function is both in the *auth.js* and *user.js* files, and what it does is, among other things, is check if the username is contained in the password. The particularity is that it uses a *String.prototype.match()* function. Whatever is input into *match*, JavaScript turns into a Regular Expression object, and that is the vulnerability.

Exploit + EXTRA

We set a regular expression as the username, e.g. `^(a+)+$`, then a long repetitive string as the password, e.g. `aaaaaaaaaaaaaaaaaaaaaX`, using at least 50 characters. The application will crash because the regex engine gets stuck trying to calculate all the exponential ways to group the a's before failing on the X, the Node.js event loop is completely blocked. Because Node.js uses a single-thread event loop, the server's CPU will be blocked.

We also found that there is potentially dangerous input handling in the `/search` GET call. It takes the URL parameter as a regular expression without sanitizing it first.

We tried exploiting it by creating a thread with the really long string from the main vulnerability and then using the same regular expression as the URL parameter. However, the ReDoS does not happen because MongoDB sanitizes suspicious RegEx expressions.

Mitigation

To mitigate the username and password vulnerability, the solution is to change the *match* function to *includes* as it does not turn the input into a regular expression.

To mitigate the extra vulnerability, we would need to strip the input of special RegEx characters.

3 Web Side Channels + EXTRA Vulnerability

Description

After a few trials on the web interface, we found two related storage side channels. The first side channel is in the registration flow. The `/signup` endpoint reveals whether a submitted license plate is already registered by another user. By submitting registration attempts with fresh random usernames and guessed license plates, an attacker can determine which license plates are stored in the database. The second side channel is in the parking flow. The `/park` endpoint reveals if a license plate is already parked in that location and gives a distinguishable response. This leaks whether a specific license plate is parked at a specific location. An extra vulnerability that we found is that users have control to license plates that are not theirs. Together, the two side channels and the extra vulnerability allow an attacker to first discover registered license plates and then infer where one of those plates is parked.

Registration Side Channel

The exploit sends registration requests with random usernames and candidate license plates. It then checks whether the returned HTML contains the error text "License plate already registered by other user". If this text appears, the script concludes that the tested plate is registered so the attacker can use it further.

However, one limitation is that this exploit is not safe for arbitrary scanning: if the guessed plate is not already registered, the application may create a new account. Therefore, in order to not modify the databases, we used it only with plates that were already known to be registered.

Parking Side Channel + EXTRA

A normal `/park` request with a valid `mintime` changes the database when the car is not already parked at the tested location. We observed this during testing: the script caused the car to become parked at every location that returned False. To avoid this, we used `mintime = -1`, since the endpoint checks whether the car is already parked before it validates the minimum parking time. The exploit logs in as Alice, scrapes the location IDs from the homepage, and then submits the target plate to `/park` with each location ID and `mintime = -1`. By combining this with the vulnerability, an attacker can check where each car is parked by simply adding the stolen license plate into their list.

Fixes

Registration Side Channel

The application should not reveal whether the username or license plate specifically caused registration failure. It should return a generic message such as “Registration could not be completed with the provided information.” The application should also rate-limit registration attempts to make enumeration harder.

Parking Side Channel + EXTRA

The application should not trust `licensePlate` from the request body. Instead, it should use the logged-in user’s registered plate from the database `let licensePlate = req.user.plate;` or explicitly check ownership before any parking lookup. The error messages should also be generic. Instead of revealing that the car is already parked in the selected location, the endpoint should return something like: “Unable to complete parking request.”, as this prevents attackers from distinguishing whether the request failed because the car was already parked, the time was invalid, or the user was unauthorized.

4 XSS + EXTRA Vulnerability

We approached this task by trying to see which fields render raw HTML. By following our intuition we searched the `thread.hjs` file and we saw that the thread body is rendered with triple braces. This disables HTML escaping and causes stored thread content to be interpreted as HTML. Therefore, a user can create a thread with HTML/JavaScript markup in the body, and that markup is rendered when another user views the thread. We also tested this ourselves with a very simple html `<b>XSS_TEST</b>` and saw that when the thread was posted the text was bold. We then tried to insert a simple JavaScript snippet `<script>alert("XSS worked")</script>` but that was unsuccessful due to the filtering that takes place in the `forum.js` file, which removes or breaks lowercase `<script>` tags. As a result, the stored output only contained the remaining part of the tag rather than a valid script element. However, the filter is incomplete because it is case-sensitive. Browsers treat HTML tag names case-insensitively, so `<ScRiPt>` is still interpreted as a script tag by the browser. So, we bypassed the filter using a mixed-case script tag.

CSP + CSP Bypass

Content Security Policy (CSP) is a browser security mechanism that restricts which resources a page is allowed to load and execute. In particular, CSP can restrict JavaScript execution by controlling whether inline scripts are allowed and which origins external scripts may be loaded from. A strong CSP can reduce the impact of XSS because even if an attacker injects HTML into the page, the browser may refuse to execute the injected JavaScript. Formerly uses the following relevant CSP directive:

```
script-src 'self' use.fontawesome.com ajax.googleapis.com cdnjs.cloudflare.com
```

This policy blocks inline JavaScript, like the snippet that we tried and event-handler attributes since the policy contains `script-src-attr 'none'`. However, the CSP still allows scripts from `'self'`, meaning scripts served from the same origin as the application are trusted. This is where the extra vulnerability that we found was useful. This policy blocks inline JavaScript, like the snippet that we tried and event-handler attributes since the policy contains `script-src-attr 'none'`. However, the CSP still allows scripts from `'self'`, meaning scripts served from the same origin as the application are trusted. We initially uploaded the malicious file with injection ourselves in the `javascript` folder. When creating a thread with the body `<ScRiPt src="/javascript/attack.js"></ScRiPt>`, we got the alert, proving that we successfully bypassed the CSP. But since an attacker shouldn't have access to the code folders, we tried to find a way where they could somehow upload a malicious file. This is where the extra vulnerability that we found was useful. Since the profile picture upload lacks sufficient validation, we were able to upload a malicious file containing our JavaScript and it was served from the same origin. The uploaded file was then served from the same origin at `/images/profileImages/alice`. So, by creating a thread whose body contained a mixed-case external script tag, `<ScRiPt src="/images/profileImages/alice"></ScRiPt>`, we were able to perform the XSS attack. This bypasses two defenses at once. First, the mixed-case `<ScRiPt>` bypasses the application's naive script-tag filter, because browsers treat HTML tag names case-insensitively. Second, the external `src` bypasses the inline-script restriction in CSP, because the script is loaded from the same origin and `script-src 'self'` allows same-origin scripts. Thus, the CSP does not fully prevent exploitation. It blocks simple inline scripts, but it can be bypassed when attacker-controlled JavaScript is hosted on the same origin.

Impact

Since this vulnerability is a stored XSS vulnerability, the malicious payload is stored in the thread body and any user who opens the affected thread executes the attacker-controlled JavaScript in their browser. The script executes under the Formerly origin. Therefore, it can read and modify page content visible to the victim and can send authenticated requests as the victim. For example, an attacker-controlled script could read sensitive information from the page or send the visible page contents to an attacker-controlled endpoint. If the application contains forms or hidden tokens, the script could read those values and use them in further requests. The attacker could also create posts, replies, or other application actions as the victim by sending same-origin `fetch` requests.

Fix

We think that the vulnerability should be fixed in several steps. First, the thread body should not be rendered with triple braces as it is right now, as this renders the thread body as raw HTML. Thus, it should be replaced with escaped output `{{ thread.body }}`. This causes characters such as `<`, `>`, and quotes to be escaped before being inserted into the page. For example, `<script>` would be displayed as text instead of becoming an actual script element. Moreover, if the application intentionally wants to allow some formatting in posts, it should not use raw HTML directly. Instead, it should use a whitelist-based sanitizer.

Second, the profile picture upload should validate the uploaded files. The application should only allow real image types, such as PNG or JPEG and reject any other file type. The server should check both the file extension and the actual file content. Moreover, the upload directory should not be a trusted script source. Ideally, user uploads should be served from a separate domain that is not allowed by `script-src`.

Third, the CSP should be strengthened. The current policy allows `script-src 'self'`. This is too broad if attackers can upload files to the same origin. A safer CSP would use nonces or hashes for trusted scripts and would not allow arbitrary same-origin uploaded files to execute as scripts. With a nonce-based CSP, each legitimate script tag generated by the server receives a fresh random nonce:

```
<script nonce="RANDOM_NONCE" src="/javascript/main.js"></script>
```

. The browser only executes scripts with the correct nonce. An attacker-injected script tag such as the one we used wouldn't have the right nonce and thus will not be executed. It is not possible to guarantee that a web application will prevent every possible malicious script in all circumstances, but a secure design can make this attack ineffective.

5 RCE

5.1 Prototype Pollution Pattern questions

Review the code of formerly for prototype pollution patterns.

As the lab assignment said, we searched for elements in the code where the user had control over the two parts of a variable assignment. We saw this in the `/upload/users` route, where a user-uploaded JSON file is parsed and the data is passed into a recursive `merge(user, item)` function. Because the attacker controls the item object, they can pass `__proto__` as a key, which the merge function improperly assigns to the global `Object.prototype`, a property most objects in JavaScript have.

What request should an attacker send to trigger an execution of the prototype pollution pattern?

An attacker should send a POST request with a JSON file to `/upload/users`. The JSON payload must contain an object where the `lcUsername` is an existing user, for example `admin`, to make sure the code execution reaches the `merge()` function.

Should an attacker be authorized in the system to execute code subject to prototype pollution?

No. We sent the payload with `curl` without the need to provide session cookies.

How can you verify that you achieve prototype pollution exploitation, e.g. by using a debugger with VSCode or similar?

We verified it by polluting *toString*, which is a method widely used. The server immediately crashes and returns a `TypeError` saying that the *Object.prototype.toString.call* is not a function. This is our DoS vulnerability.

5.2 DoS questions

What payload do you need to cause Denial of Service by prototype pollution?

```
[
  {
    "lcUsername": "admin",
    "__proto__": {
      "toString": "boom"
    }
  }
]
```

Mitigation

The mitigation would be to filter out dangerous JavaScript like *proto*, *constructor*, or *prototype* from the *merge* function. This would also mitigate the RCE attack.

5.3 RCE questions

Which code of formerly can be affected by prototype pollution and lead to RCE?

The *eval()* call is located in *passport.js*, which is used when a user logs in. When login fails, the *eval()* is executed with a function as input. That is our target.

What request should an attacker send to trigger a gadget execution? What property should be polluted to achieve the gadget execution and pass an attacker-controlled value to *eval()* or *new Function()*?

userAutoCreateTemplate is the property we need to pollute. Because *eval()* executes code if it is fed with it, we need to pollute this property. The code we will try to execute is the reverse shell code we get from the *nodejshell.js* file. First, we set up a listener with *nc*. Then, we send a POST request to the DoS endpoint, but with a payload that pollutes *userAutoCreateTemplate*. The server will crash, and immediately, we send another POST request to the */login* endpoint with a user that does not exist. The *eval()* will execute because of the pollution, and we'll get a reverse shell.

6 Contributions

Ana took care of Side Channels and XSS, and Aurora took care of ReDoS and RCE.